

Forward-Deployed Engineering Exercise

What this is

Bedrock assembles small teams of forward-deployed engineers who work directly with clients, make sense of ambiguous operational problems, and ship production software quickly. This exercise is a compressed version of that job. You will be handed a real-shaped problem: a client, their messy operational data, and a mandate that is deliberately broader than a spec.

There is no single intended answer. The data contains real signal, real noise, and real contradictions, exactly as client data does. We are evaluating how you decide what matters, what you build, and how you reason about it, not whether you found a hidden checklist.

Everything in this package is fictional. The client, its customers, all people, addresses, and email domains are invented for this exercise. Any resemblance to real companies or people is coincidental.

The client

Sego Industrial Water is a water treatment service company in Salt Lake City. Its technicians drive routes across Utah servicing boilers, cooling towers, and closed loops at about two dozen industrial and institutional facilities: hospitals, food plants, a data center, hotels, manufacturers. Technicians test water chemistry, dose chemicals, record observations, respond to complaints, and open work orders.

The company is healthy but its information is fragmented: spreadsheets, free-text field notes, lab reports in a different format, a shared email inbox, an inventory sheet, and an internal guidelines binder. Management believes problems are being recognized too late, and recent events support that belief. They can articulate the pain precisely but not the solution. Read their own words in `organization_context.pdf`.

Your assignment

Using the materials provided, build the most useful working system you can for this company. The system should help someone responsible for operations understand what deserves attention

and take appropriate action.

That is the whole spec. Determining the primary user, the most important problem, and the right product is part of the exercise. You may build whatever form of software serves the problem best. Choosing well matters more than building big.

Two hard requirements sit inside that freedom:

- 1. Your system must work against the provided data as it actually is, not against a cleaned version you curate by hand.
- 2. A second batch of operational data is included under `materials/new_data_batch/`. Your system must be able to incorporate it without being manually reconstructed. It contains new records and some corrections to earlier records. If your package arrived without this folder, it will be sent to you during the exercise window.

The materials

File or folder	Contents
<code>organization_context.pdf</code>	The company, team, process, and pain points, in their words
<code>treatment_guidelines.pdf</code>	Internal reference: control limits, interpretation rules, escalation policy
<code>customers.csv</code>	Customers, facilities, contracts, tiers, contacts
<code>systems.csv</code>	Water systems: type, capacity, program, criticality
<code>service_visits.csv</code>	Six months of technician visits
<code>water_readings.csv</code>	Water chemistry readings: field, lab, and building automation
<code>work_orders.csv</code>	Work orders with status, costs, resolutions
<code>chemical_inventory.csv</code>	Warehouse and truck inventory snapshot
<code>technician_notes/</code>	Unstructured field notes, various formats and styles
<code>customer_communications/</code>	Service inbox export: complaints, requests, internal mail
<code>new_data_batch/</code>	Early September data in new export formats, with corrections

The data is intentionally unpolished. Identifiers, units, and quality vary across sources. Some records contradict each other. Deciding what to trust, what to reconcile, and what to ignore is part of the work.

Constraints

Time. You have four calendar days from receiving this package. We expect 12 to 15 hours of actual work. Do not spend more; part of the exercise is choosing what not to build. Tell us roughly where your hours went.

Cost. You are not expected to spend any money. Use free tiers, local tools, open-source software, and open-weight models wherever a paid service would otherwise appear. We do not evaluate candidates negatively for choosing a free or open-weight alternative over a premium product. If a paid tool or model would materially improve a production implementation, say so in your architecture notes instead of buying it. Do not include secrets, personal credentials, or paid API keys in your submission.

Stack. Any language, framework, database, model, or deployment approach. Technology choices should be justified by the problem, the constraints, and a plausible production path at a client like this one, not by familiarity alone.

AI. Use modern AI models, coding agents, and development tools as much as you like; we build with these tools daily and we are interested in how well you direct them. You will not be penalized for using AI heavily, and the work is not graded on unassisted effort. Two things remain yours regardless of tooling: every architectural, product, and implementation decision in the submission, and the ability to explain, modify, and debug all of it. Inside the product you build, AI is optional. A conventional software solution is completely acceptable if AI does not improve the workflow. Where you do use AI in the product, explain why, name its failure modes, and show how consequential outputs remain reviewable by a human.

Deliverables

Submit a repository (link or archive) containing or pointing to all of the following.

1. The working system, with clear instructions that get an evaluator from clone to running in minutes. Local execution is fine; hosted is fine; neither is worth more points.
2. `WRITEUP.md`, concise: what you chose to build and why, the primary user you designed for, your major assumptions, important limitations and risks, unresolved questions, your technology and architecture choices, any AI usage inside the product and its safeguards, and what you would build next if this were deploying with a real customer. No slide deck, no formal report. Plain writing.
3. `FIELD_NOTES.md`, an engineering log, not an essay: your initial read of the problem, what you prioritized and why, assumptions made along the way, approaches you considered and rejected, at least one thing that went wrong or failed, what you learned from investigating the operational data, and what you would change with another week.

4. `AI_USAGE.md` : the models, coding agents, and major development tools you used and for what; one concrete example of model-generated output you rejected, corrected, or substantially redesigned, and why; the parts of the system that most need human or production validation; and your approximate personal time spent. We do not want prompt logs or transcripts.
 5. A demonstration video, 8 to 10 minutes, screen recording of the actual working system (a slide presentation alone is not sufficient), covering: the primary user and operational problem you chose; the working product; one important operational finding and the evidence behind it; the second data batch being incorporated; one important limitation or failure mode; and what you would build next.
 6. A commit history that reflects how the work actually developed. It does not need to be pretty. It needs to be real and reasonably legible.
-

How we evaluate

We read the writeup and field notes first, then run the system, then watch the video. We evaluate submissions across the following dimensions:

- Problem framing and prioritization
- Technical execution
- Product judgment
- Data modeling and reconciliation
- Adaptation to the second data batch
- Handling of uncertainty and contradictory information
- Reliability, security, and maintainability
- Communication and handoff
- Judgment around AI
- Understanding of operational deployment

Some guidance on what those words mean to us. Finding every anomaly in the data is not the goal and is not scored; coherent prioritization of what matters, and a defensible line from evidence to recommendation, score higher than a long list of detections. A smaller system that a named user would actually adopt beats a larger one that demos well. Work another engineer could pick up on Monday beats work that needs its author present.

After you submit

We will schedule a 40 minute technical review: a walkthrough of your system, a code review of parts we select, a discussion of your data and product decisions, and a short live modification exercise in your own codebase. You may use your normal tools during the live portion, including AI assistance. It is a working session, not a quiz; we want to see how you investigate, direct your tools, validate changes, and communicate while doing it.

Logistics

Send your repository link (or archive) and video link to your Bedrock contact by the agreed deadline. If anything blocks you from working (a corrupted file, an inaccessible link), reach out immediately. Questions about the problem itself are welcome but not required; expect the kind of answers a busy operations director would give, and treat silence on a detail as the exercise working as intended. Document your assumption and keep moving.